Writing your own functions

🎯 Today's goal

By the end of this hour you will:

- create your own **functions** with `def` and call them
- pass values in with **parameters**, including ones with a default value
- understand the difference between `return` and `print`
- return several values at once, and know which variables a function can see
- write functions that work on whole lists, and sort with your own rule

🤔 Why this matters

You've already used lots of functions: `print()`, `len()`, `sum()`, `round()`. Someone wrote them once, and now everyone can use them without knowing what's inside. Today you'll write your own.

A function is like a **recipe card**. You write the steps once and give the card a name ("pancakes"). Whenever you want pancakes, you just say "pancakes" and maybe "for 4 people", instead of repeating every step. In data work you'll write functions like `clean_price()` or `average()` and use them on thousands of rows. And when a step needs fixing, you fix it in one place.

🧩 Your first function

You **define** (create) a function with the word `def`, a name, round brackets and a colon. The steps go in an indented block, just like with `if` and `for`. Defining a function doesn't run it yet. You run it by **calling** it: its name followed by brackets.

```
def say_hello():
    print("Hello!")
    print("Welcome to Day 6.")

say_hello()      # call it once...
say_hello()      # ...and again
```

```
Hello!
Welcome to Day 6.
Hello!
Welcome to Day 6.
```

A function becomes useful when you can give it information. The names inside the brackets are called **parameters**: variables that get their value when the function is called. The value you pass in is called an **argument**.

```
def greet(name):
    print(f"Hello, {name}! Ready for some data?")

greet("Sara")       # name becomes "Sara"
greet("Omar")       # name becomes "Omar"
```

```
Hello, Sara! Ready for some data?
Hello, Omar! Ready for some data?
```

↩️ Giving back a result: `return`

Most functions **calculate** something and hand the answer back, like `len()` hands back a number. That's what `return` does. The returned value can be stored in a variable, printed, or used in more maths:

```
def c_to_f(celsius):
    fahrenheit = celsius * 9 / 5 + 32
    return fahrenheit

body = c_to_f(37)          # the returned value goes into body
print(body)
print(c_to_f(100))
print(c_to_f(0) + 10)      # a returned value can be used in maths
```

```
98.6
212.0
42.0
```

`return` also **ends** the function straight away: any lines after it are skipped. Now combine it with yesterday's loop to convert a whole (made-up) week:

```
week_c = [21.5, 23.0, 19.8, 25.1, 27.3]

week_f = []
for t in week_c:
    week_f.append(round(c_to_f(t), 1))

print(week_f)
```

```
[70.7, 73.4, 67.6, 77.2, 81.1]
```

🖨️ `print` is not `return`

This trips up almost everyone. `print` only **shows** a value on the screen, for a human to read. `return` **hands it back** to your code, so the program can keep using it. Look what happens with a function that prints instead of returning:

```
def c_to_f_printer(celsius):
    print(celsius * 9 / 5 + 32)     # shows the answer, but doesn't return it

result = c_to_f_printer(37)         # the number appears on the screen...
print("result is:", result)         # ...but result is empty!
```

```
98.6
result is: None
```

A function without `return` gives back `None`, Python's "nothing". If you later try `result + 10`, you'd get a `TypeError`. Rule of thumb: **functions should `return` their answer**; print it outside, when you want to see it.

🎛️ Several parameters and default values

A function can take several parameters, separated by commas. You can give a parameter a **default value** with `=`; it's used whenever the caller doesn't pass that argument. Here's a made-up sales tax of 20%:

```
def price_with_tax(price, tax_rate=0.2):
    return round(price * (1 + tax_rate), 2)

print(price_with_tax(50))              # tax_rate uses its default, 0.2
print(price_with_tax(50, 0.05))        # pass a different rate
print(price_with_tax(50, tax_rate=0))  # you can name the argument, which is easier to
read
```

```
60.0
52.5
50
```

If you forget an argument that has **no** default, Python tells you exactly which one is missing:

💡 This example has a mistake on purpose. Run it and read the error.

```
price_with_tax()
```

> ⚠️ **TypeError**
>
> You combined things that don't fit together, like text + a number.
>
> ```
> Traceback (most recent call last):
>   line 1, in <module>
>     price_with_tax()
>     ~~~~~~~~~~~~~~~^^
> TypeError: price_with_tax() missing 1 required positional argument: 'price'
> ```

missing 1 required positional argument: 'price': the function needs a price, and we didn't give one. (**Positional** means the arguments are matched by their order.)

📊 Functions that work on lists

A parameter can hold anything, including a whole list. And a function can use a loop and `if` inside it. Here is a function that counts the values above a limit:

```
def count_above(values, limit):
    count = 0
    for v in values:
        if v > limit:
            count += 1
    return count

sales = [120, 95, 140, 80, 160, 210, 175]     # made-up café sales
print(count_above(sales, 150))
print(count_above(sales, 100))
print(count_above(week_c, 22))                # works for any list
```

```
3
5
3
```

A function can also return **several values**: separate them with commas. Python packs them into a **tuple**, which is like a list that can't be changed, written with round brackets. You can unpack a tuple straight into several variables:

```
def low_high(values):
    return min(values), max(values)

result = low_high(sales)
print(result)               # a tuple: (80, 210)

low, high = low_high(sales) # unpack: the first value goes into low, the second into
high
print(f"Sales ranged from {low} to {high} dollars")
```

```
(80, 210)
Sales ranged from 80 to 210 dollars
```

🔒 What happens inside stays inside

Variables created inside a function are **local**: they exist only while the function runs, and they don't touch variables with the same name outside. That's a good thing: you can use a name like `total` in a function without worrying that it breaks the rest of your code.

```
total = 1000          # a variable outside any function

def add_up(values):
    total = 0         # a different, local total
    for v in values:
        total += v
    return total

print("Inside the function:", add_up([1, 2, 3]))
print("Outside, unchanged:", total)
```

```
Inside the function: 6
Outside, unchanged: 1000
```

Good habit: give a function everything it needs through its parameters, and get everything out through `return`. Then it works the same wherever you use it.

A text in triple quotes `"""…"""` right under the `def` line is a **docstring**: a short description of what the function does. Python stores it, and `help()` shows it. Good functions also **call other functions**:

```
def describe_temp(celsius):
    """Return a sentence describing a temperature in °C and °F."""
    fahrenheit = c_to_f(celsius)             # a function calling another function
    if celsius >= 30:
        feeling = "hot"
    else:
        feeling = "pleasant"
    return f"{celsius}°C is {fahrenheit:.1f}°F, {feeling}"

print(describe_temp(31))
print(describe_temp(18))
help(describe_temp)
```

```
31°C is 87.8°F, hot
18°C is 64.4°F, pleasant
Help on function describe_temp in module __main__:

describe_temp(celsius)
    Return a sentence describing a temperature in °C and °F.
```

🔀 Sorting with your own rule

On Day 3, `sorted()` put numbers smallest-first and words alphabetically. You can give it a different rule with `key=`: a function that is applied to every item, and the items are sorted by what it returns. Any function works, even `len`:

```
cities = ["Rome", "Ho Chi Minh City", "Lagos", "Montevideo"]

print(sorted(cities))              # alphabetical
print(sorted(cities, key=len))     # shortest name first
```

```
['Ho Chi Minh City', 'Lagos', 'Montevideo', 'Rome']
['Rome', 'Lagos', 'Montevideo', 'Ho Chi Minh City']
```

To sort the made-up patients from Day 3 by age, we need a tiny function that takes one patient and returns their age. For throwaway functions like that, Python has a one-line form called a **lambda**: `lambda p: p["age"]` means "a function that takes `p` and returns `p["age"]`".

```
patients = [
    {"name": "Amir", "age": 34},
    {"name": "Bea",  "age": 58},
    {"name": "Chen", "age": 25},
]

by_age = sorted(patients, key=lambda p: p["age"])
for p in by_age:
    print(p["name"], p["age"])

oldest = max(patients, key=lambda p: p["age"])     # max and min accept key= too
print("Oldest:", oldest["name"])
```

```
Chen 25
Amir 34
Bea 58
Oldest: Bea
```

⚠️ Common mistakes

- Defining a function but never calling it. Nothing happens until you write `name(...)`.
- Calling it without brackets: `say_hello` just names the function; `say_hello()` runs it.
- Printing instead of returning, so the result is `None`.
- Putting `return` inside a loop by accident: the function ends during the first round.
- Forgetting an argument → `TypeError: … missing 1 required positional argument`.
- Expecting a variable made inside a function to exist outside it. Return it instead.

✅ Recap

- `def name(parameters):` + an indented block creates a function; `name(arguments)` calls it.
- `return value` hands the answer back and ends the function. Without it, you get `None`.
- Defaults: `def f(price, tax_rate=0.2):`. Named arguments: `f(50, tax_rate=0)`.
- `return a, b` returns a tuple; unpack it with `x, y = f(...)`.
- Variables inside a function are local. A docstring describes the function.
- `sorted(items, key=len)` or `key=lambda p: p["age"]` sorts by your own rule.

🏋️ Exercises

Write your answer in the space under each exercise. Try it on the computer too, by opening this day's `index.html`. Solutions are in the answer key at the end.

How today's checker works: it calls your function with several different values and looks at what it **returns**. So use `return`, not `print`. In the starting code, `pass` means "do nothing yet"; replace it with your code.

1. Celsius to Fahrenheit

Complete the function `c_to_f(celsius)` so it **returns** the temperature in Fahrenheit: multiply by 9, divide by 5, then add 32.

```
def c_to_f(celsius):
    pass   # replace this line


print(c_to_f(100))    # should show 212.0
```

Hints:

1. The formula is `celsius * 9 / 5 + 32`.
2. Hand the answer back with `return`, indented inside the function: `return celsius * 9 / 5 + 32`

2. Average of a list

Write a function `average(values)` that returns the average of a list of numbers. If the list is **empty**, it should return `0` instead of crashing (dividing by zero is not allowed).

```
def average(values):
    pass   # replace this line


print(average([18.5, 20.1, 22.3]))   # about 20.3
print(average([]))                   # 0
```

Hints:

1. The average is `sum(values) / len(values)`.
2. Check for an empty list first: `if len(values) == 0:` (or `if not values:`) → `return 0`
3. After that `if`, `return sum(values) / len(values)`

3. Grade function

Write a function `grade(score)` that returns a letter, using Day 4's `if` / `elif`:

score	returns
90 or more	`"A"`
80 to 89	`"B"`
70 to 79	`"C"`
50 to 69	`"D"`
below 50	`"F"`

```
def grade(score):
    pass   # replace this line


print(grade(74))    # should show C
```

Hints:

1. Check the biggest band first: `if score >= 90:` → `return "A"`
2. Then `elif score >= 80:` → `return "B"`, and so on down to 50.
3. Finish with `else:` → `return "F"`. Each `return` is indented inside its band.

4. Count the vowels

Write a function `count_vowels(text)` that returns how many vowels (`a e i o u`) are in a text. Capital letters count too: `count_vowels("Data")` is `2`.

Good to know: a `for` loop over a string goes through it one character at a time, so `for letter in "Data":` gives `"D"`, `"a"`, `"t"`, `"a"`. And `in` works on strings too: `"a" in "aeiou"` is `True`.

```
def count_vowels(text):
    pass   # replace this line


print(count_vowels("Data Science"))   # should show 5
```

Hints:

1. Start a counter at 0 inside the function, loop over the letters, and return the counter at the end (after the loop, not inside it).
2. To make capitals count, compare the lowercase letter: `if letter.lower() in "aeiou":`
3. `count = 0` → `for letter in text:` → `if letter.lower() in "aeiou":` → `count += 1` → `return count`

5. Challenge: a summary function

Write a function `summary(values)` that returns a **dictionary** with three keys:

- `"min"` : the smallest value
- `"max"` : the largest value
- `"mean"` : the average, rounded to 1 decimal

For example, `summary([4, 8, 6])` returns `{"min": 4, "max": 8, "mean": 6.0}`. You may assume the list is never empty.

```
def summary(values):
    pass   # replace this line


steps = [6200, 8400, 10150, 7300, 9900]   # made-up daily step counts
print(summary(steps))
```

Hints:

1. Calculate the three values with `min()`, `max()` and `sum() / len()`, then `round(..., 1)` the mean.
2. Build the dictionary from Day 3: `{"min": low, "max": high, "mean": mean}`
3. `return {"min": min(values), "max": max(values), "mean": round(sum(values) / len(values), 1)}`

➡️ Tomorrow

Files, modules and handling errors: you'll use functions other people have already written, by `import`ing modules like `math`, `random` and `statistics`. You'll save data to a file and read it back, and learn `try` / `except`, so a bad value (like the text `"n/a"` where a number should be) doesn't crash your whole program.

🔑 Answer key

Try the exercises yourself first! There is often more than one correct answer.

1. Celsius to Fahrenheit

```
def c_to_f(celsius):
    return celsius * 9 / 5 + 32


print(c_to_f(100))    # should show 212.0
```

2. Average of a list

```
def average(values):
    if len(values) == 0:
        return 0
    return sum(values) / len(values)


print(average([18.5, 20.1, 22.3]))   # about 20.3
print(average([]))                   # 0
```

3. Grade function

```
def grade(score):
    if score >= 90:
        return "A"
    elif score >= 80:
        return "B"
    elif score >= 70:
        return "C"
    elif score >= 50:
        return "D"
    else:
        return "F"


print(grade(74))    # should show C
```

4. Count the vowels

```
def count_vowels(text):
    count = 0
    for letter in text:
        if letter.lower() in "aeiou":
            count += 1
    return count


print(count_vowels("Data Science"))   # should show 5
```

5. Challenge: a summary function

```
def summary(values):
    low = min(values)
    high = max(values)
    mean = round(sum(values) / len(values), 1)
    return {"min": low, "max": high, "mean": mean}


steps = [6200, 8400, 10150, 7300, 9900]   # made-up daily step counts
print(summary(steps))
```